Because: preregistered randomized tests of what makes a coding agent follow its instructions

Abhay Mettu

November 2026. Preprint, not peer reviewed.

Abstract

People write instruction files for coding agents on folk beliefs: give a reason, write MUST, put the important rule first, phrase a skill's description as a list of triggers. We tested these on Claude Code 2.1.303 with Sonnet 5.5 at medium effort in four preregistered randomized studies, with outcomes decided by code. Study 1 (1,000 runs, 7,072 observations on nine rules) crossed reason clauses with wording features. Its registered reading is inconclusive. The bar for calling a reason null was an upper bound under 5 points, and the real reason reached 5.7 (+0.2, 95% CI −5.2 to +5.7). That interval is for the average across these nine rules against filler, and the prediction interval for a new rule runs from −11.6 to +11.9. A placebo reason is +1.1 (−1.2 to +3.5), the Langer analogue does not reproduce, and no wording factor's average survives Holm. Study 2 (598 runs) finds that a trigger list beats one short sentence with the description in view (+17.6 points) and that an early name beats a late one at 300 skills (+51.3). The vendor documents a character budget on the skill listing, and we measured the exact rule behind it with no model judgment. Study 2b (534 runs) tests never-used skills that sort after that budget is spent. There a short description beats a long trigger list by 29.8 points, and in trigger form by 29.4, and at 50 skills, near the ceiling, that form costs under 10 points (+2.1, −2.5 to +6.7). Study 2c (384 runs) changes only the budget. Raising it until every description fits lifts the trigger list by 69.8 points (+53.7 to +85.8) and puts it 16.7 above the short description (+0.7 to +32.7), with an interaction of +40.6 (+22.7 to +58.6), so raising the budget directly reverses the reversal. The wording of a skill's description matters through whether it reaches the model.

1 The question

This paper tests whether the wording of an instruction file changes what a coding agent does. Files such as `CLAUDE.md` and `AGENTS.md` are where a repository tells an agent how to work, and the people who write them follow folk rules: give a reason, write MUST, put the important rule first, keep the file short, write a skill's description as a list of triggers or as "ALWAYS invoke". A census of public repositories finds a median instruction set of 818 words, finds that 74.0% of directives give no reason, and says of the causal question, "Whether reasons or emphasis change what agents do remains open" [19].

The folk rules matter because they decide where effort goes. If a reason clause raises compliance, a team should explain every rule to the agent. If it does not, that writing buys nothing the agent acts on, and the effort belongs elsewhere. Skills add a second problem, which the vendor documents [2]. A well-written description does nothing if the CLI drops it from the listing before the model sees it.

We ask four questions. Does a reason clause raise compliance, and does a reason with no content work as well as a real one? How much does each folk wording feature move compliance on the same rules? When does a skill's description decide whether the skill loads? Is the listing budget what makes a short description beat a long one? We answer them with four registered studies, frozen at the git tags `prereg-study1`, `prereg-study2`, `prereg-study2b` and `prereg-study2c`. Study 2c answers the last question directly: at 300 skills, moving the budget and nothing else reverses the reversal, so the wording of a description matters there through whether it reaches the model. Section 9 lists every deviation, including one rewrite of the git history. All claims are about Claude Code 2.1.303 and Sonnet 5.5 at medium effort, on these rules and tasks. We make no claim about other agents, models or CLI versions, about what a reason does for a person reading the file, or about how a model processes a reason.

2 Related work

Vendor documentation. The vendor's prompting guide says, "Providing context or motivation behind your instructions, such as explaining to Claude why such behavior is important, can give Claude more information about your goals and help it deliver more targeted responses", with a text-to-speech example that ends, "Claude can generalize from the explanation" [1]. Our design scores literal compliance only, so that claim about generalizing is not tested here. The vendor's skills page says that the listing "always contains every skill name", that Claude Code "drops some descriptions to fit the listing's character budget", which "scales at 1% of the model's context window", and that it drops them "starting with the skills you invoke least" [2]. So the limit is documented, and what we add is the exact rule behind it, including the skip and the tie order, and the size of its consequence. Both are current pages, not pinned to 2.1.303, and the same 1% is in the code of the CLI we ran (`docs/LISTING-MECHANISM.md`).

Instruction-file censuses and audits. Wang et al. [16] find that a rule raises the rate at which an agent takes a required action by +0.23 on average across four agents, and that on four open models a prose-only revision moves compliance by +0.017, inside an equivalence margin of 0.08. Zhang et al. [22] report that random rules match curated ones (both 63.8%). Zhu et al. [23] find that repository precedent beats an agent's default in 359 of 360 trials, and that a written priority rule succeeds in 59 of 60 trials against 38 of 60 for a code example. Xu and Wu [17] find that 90.1% of choices follow a stale file over the code, and one precedence sentence moves code-following from 4.2% to 91.4%. A self-published book reports 39 trials on Claude Code 2.1.246, in which a banned command was attempted in 0 of 22 with the rule written in and 17 of 17 without, and its public description lists no reason arm [18]. None of these randomizes a reason.

The closest work is McMillan [11], who ran 1,650 Claude Code sessions varying file size from 25 to 500 lines, position, architecture and conflict, on one trivial rule always justified as an audit requirement. All four came out null after correction, and its intervals rule out effects above about 6 points on position and architecture. Reason was held constant there. Position and length in our Study 1 repeat those manipulations on nine other rules.

Skill loading. Liu et al. [10] own the library-size result. With 1,000 irrelevant skills, loaded-skill recall falls from 73.6% to 27.4%. On a separate 10-task subset, with one run per task and model, masking distractor descriptions raises recall from 18.3% to 63.3%. They never vary the target's own description. In a preregistered study of two OpenAI models, Wang [15] find that praise in a tool description raises its pick rate by about 43 points. Spence [14] runs about 250 sandboxed Claude Code calls, testing hooks and not description wording. Zhang [21] shows that conditioning on a trigger inside the treated run does not identify an invocation effect, which is why we count loads at any point in a run and score intention to treat.

Prompt emphasis and examples. Zhan et al. [20] show that wordings a verifier treats as equal shift compliance in chat replies, from 54.9% for the positive form to 33.8% for AVOID. They have no reason factor and no instruction file. Lauziere et al. [9] fit regressions on crossed prompt components, the nearest factorial method, on two open models. Razmara et al. [12] find that the worked examples inside skills drive which rules get followed when two same-job skills are loaded together, on seven models. One informal practitioner test, at 9 to 10 runs per strategy on one project with Sonnet, reports that a consequences strategy did not improve adherence [6].

The Langer line and its critics. Our analogy comes from Experiment 1 of Langer et al. [8], 120 adults at a copier. For a 5-page favor, compliance was .60 with no information, .93 with a placebic reason ("because I have to make copies") and .94 with a sufficient one. For a 20-page favor it was .24, .24 and .42. The small-favor result rests on about 15 people per cell. Folkes [4] reports of two further studies that "Neither study replicated Langer et al.'s results." We treat this as an analogy to a contested paradigm and not as a replication target.

What is new and what is not. To our knowledge, no preregistered randomized test of a reason or a placebo reason on a real coding agent has been published. That statement rests on 14 searches on 2 November 2026, logged with their queries in `docs/REVIEW-3-final.md`. Practitioner tests exist and this field moves weekly, so a missing hit is not proof. Position and length repeat McMillan [11], library size belongs to Liu et al. [10], and the listing limit belongs to the vendor. Our contribution is the combination: reason, placebo, modal, emphasis and example crossed over nine rules on one real agent, and the target skill's own description and the listing budget randomized on real Claude Code.

Table 1: Factors of Study 1. Census 10th and 90th percentiles of file length are about 160 and 3,010 words [19].

Factor	Levels	Set per
REASON	none; filler; placebo; norm; real (Table 2)	rule
MODAL	plain imperative; "You must"	rule
PREFIX	none; `IMPORTANT:`	rule
CASE	normal; operative word in capitals	rule
EXAMPLE	absent; present ("For example: break a long call after its opening bracket.")	rule
POSITION	from the shuffle; top, middle or bottom third of the file by word count	rule
LENGTH	short, mean 514 words (450 to 569); long, mean 3,526 words (3,427 to 3,583)	run
PRESSURE	none; a task sentence asking for speed and a minimal change	run

Table 2: What each REASON arm adds to "Keep every line of Python you write to 60 characters or fewer", with mean ratings out of 5.

Arm	Text	Information	Obligation
none	(nothing)		
filler	"This note is one of several notes kept here in this section."	1.0	1.0
placebo	"because that is just how this particular part of the project is"	1.0	1.8
norm	"because that is how we do things here in this repository"	1.0	2.9
real	"because the on-call team reads code on phones when something breaks at night"	4.9	2.2

3 Study 1: design

Why focal rules. Compliance sits at the ceiling for most rules, and wording cannot move a rule already followed. Of 59 candidates (`docs/POOL.md`), 41 were dropped at the ceiling in pilots, and that share is itself a finding. The pool rule required shown compliance within 0.10 to 0.95 and no-rule compliance at or below 0.50, computed on compliance pooled across arms and never on a contrast. Manual steps, taken without reference to arms, dropped two rules and reworded two so that each says what its checker measures. Nine focal rules remained (Table 10). All claims are about them, which means rules an agent lapses on and not rules in general. Five more stay in the files as ballast and are reported descriptively only.

Split-plot randomization. Each run is one `claude -p` on one task. Its `CLAUDE.md` holds 11 of the 14 rules, shuffled among inert filler rules, and every shown rule's wording is drawn independently and uniformly (Table 1). Rules and tasks are fully crossed over 2 synthetic Python repositories with 3 tasks each. The assignment is committed row by row in `data/study1/manifest.jsonl`, with the seed derived from the study name. The batch aborts on any Claude Code version other than 2.1.303 or any model other than `claude-sonnet-5-5`. Runs have six tools, at most 40 turns and a 600 s timeout. Claude Code's own preamble ("IMPORTANT: ... you MUST follow them") wraps the file in every run, confirmed from captured requests, so every emphasis effect here is marginal to it.

Why filler is the comparator. A clause changes a rule's length as well as its meaning, and length alone moved compliance by 6.7 points in Zhan et al. [20], so reason versus no clause confounds the two. The filler arm adds a non-reason sentence of matched length and is the primary comparator. Filler might not be inert, since calling a rule "one of several notes" could demote it, so the reading also requires an advantage over no clause.

Clause manipulation check. Eight blind raters (Haiku 5.5, no arm labels) scored each clause out of 5 for specific information and for how strongly it asserts obligation (Table 2, `results/clause_check.json`). Real clauses carry information and the others do not. Norm carries some obligation, which is why it is a separate arm. The real reasons are invented. No voice assistant or ticket importer exists in the repositories, so this arm tests a specific plausible rationale and not a true one. Each shares no content word with its rule.

Isolation and its proof. A study of instruction following is void if the agent also sees the operator's own instructions, and every run here uses a login on a working developer machine. The harness excludes the user's instruction file, settings, hooks, memory and MCP servers, empties the environment and the home directory seen by tool calls, and rewrites the login email and OS user in a capture proxy. `make isolation` sends one prompt twice through the proxy, once isolated and once as a plain `claude -p`, and scans the requests (`results/isolation.json`). Lines of the operator's instruction and memory files found: 0 of 231

isolated and 219 of 231 plain. Skill names found: 1 of 85, the ordinary word "research", and 73 of 85. During the study a leak scan reads every request of every run, and a run that trips it is excluded and counted.

Checkers and the blind audit. Compliance is binary per rule per run, decided by code from the transcript and the final repository (`study1/rules.py`). No model judges any outcome. Every checker passes a hand-built compliant trace and fails a hand-built violating one. The primary analysis is intention to treat, so a run that never reached a rule's opportunity counts as not complied. The blind audit ran on the finished batch before any contrast was computed. It drew 100 focal observations stratified by rule and outcome, with arm labels removed and the key kept outside the repository until the audit closed. The registration names one fresh model reader. Three were used, each on about a third of the items, and every item showed the checker's verdict, as the registration allows (`PREREG.md`). So the agreement below is anchored on that verdict and is not an independent rate. The readers decided no outcome, and their job was to find checker faults.

Of the 100 items, 92 agree with the checker, 7 are unclear with the reader's lean matching the checker, and 1 disagrees. A reader also flagged one of the 92, a grouped-imports case that agrees only if a file's imports count as one list. The 7 and the 1 all turn on one question, whether a rule's "you write" reaches throwaway code that never lands in the repository. Before any contrast, we decided that no checker changes. A code-form rule scores the code the run leaves in the repository, and the Write tool rule scores the action, including a scratch file. Because one reader read an item the other way, we report a sensitivity with `write_tool` scored on repository paths only: 41 verdicts change, real becomes +0.0 [−5.3, +5.3] and placebo +0.9 [−1.4, +3.1], so the reading does not change. With about 20 items per reason arm, the audit can find a checker fault and cannot show a small difference in checker error by arm.

Estimator and tests. We fit OLS on the probability scale with run fixed effects, rule intercepts and a separate coefficient for each rule and factor level. Each contrast yields one estimate per focal rule. The reported effect is their equal-weighted mean, with a t interval across rules ($df = 8$), which treats rules as sampled stimuli [3, 7]. A studentized randomization test (2,000 permutations within rule) is the finite-population sensitivity. Run-level factors have no run fixed effects, and their standard error is by-run variance plus across-rule variance beyond its sampling part.

Family 1 is H1 (real versus filler) and H2 (placebo versus filler), two-sided, Holm across the two [5]. H3 asks whether a placebo keeps at least half the real effect, and it runs only if H1 is positive and passes. Real minus placebo is reported in all cases, with a TOST at ±5 points [13] as a registered sensitivity. Family 2 is nine contrasts under Holm. The positive control restates one unshown rule in the task prompt in about 10% of runs, and under 30 points nothing is read. `reading()` applies the registered table in the code (`PREREG.md`), so nobody applies it by hand. A null reading needs both upper bounds under 5 points, and anything the table does not list is inconclusive.

Power, realized. Simulating on the real assignment (`results/calibration.json`) gives lower figures than the registration's pilot simulation. With the Holm worst case in parentheses, a true 5-point effect has power 0.56 (0.41) at slope SD 3 and 0.34 (0.20) at slope SD 6, and a true 8-point effect has 0.91 (0.81) and 0.68 (0.53). The registered test's false positive rate at .05 with no planted effect is 0.053 over 1,000 simulations. The run-level test rejected in 0.000 of them, so it is conservative, and the long and urgent contrasts lose power to that.

4 Study 1: results

All 1,000 runs were used, giving 7,072 focal observations and overall focal compliance of 0.579. There were no errors, leaks, timeouts or turn-limit runs, and the repository's tests passed in every run. The positive control cleared its 30-point bar: 0.653 named in the prompt (n 72) against 0.038 with no rule (n 1,856).

The registered reading is "inconclusive". The code returned that word and we report it as registered (Table 3). The bar for calling a reason null was an upper bound under 5 points, and the real reason reached 5.7. H1 and H2 do not pass, so no positive reading applies, and that bound rules out the null reading.

How far the interval reaches. Averaged across these nine rules, the 95% interval against filler stops at +5.7. Against no clause the upper bound is +6.78, and leaving one rule out and refitting moves it from +3.34 to +8.58. Single rules are less certain. The real reason runs from −8.1 points on `func_lines` to +15.8 on `final_no_first_person` (`results/study1.json`). Exploratory calculations across the nine (`results/NUMBERS.md`) give Cochran's Q of 14.91 on 8 df ($p = .061$) and a DerSimonian-Laird tau of 4.41 points, so heterogeneity is not excluded. The random-effects mean is +0.15 (Hartung-Knapp −5.04 to

Table 3: Study 1 contrasts in percentage points (`results/NUMBERS.md`). Top block: family 1 and the reported reason contrasts, with Holm across H1 and H2 only. The H3 gate stayed closed because H1 did not pass, so its row decides nothing. Bottom block: family 2, Holm across the nine.

Contrast	Points [95% CI]	p	Holm p
H1 real vs filler	+0.2 [−5.2, +5.7]	.924	.924
H2 placebo vs filler	+1.1 [−1.2, +3.5]	.300	.599
H3 placebo minus half of real	+1.0 [−1.5, +3.6]	.375	
Real minus placebo	−0.9 [−5.6, +3.8]	.666	
Real vs no clause	+1.4 [−3.9, +6.8]	.549	
Placebo vs no clause	+2.4 [−1.4, +6.1]	.189	
Norm vs filler	+0.2 [−4.1, +4.4]	.927	1.000
No clause vs filler	−1.2 [−6.7, +4.2]	.622	1.000
“You must” vs plain	−0.3 [−2.2, +1.6]	.755	1.000
`IMPORTANT:` vs none	+1.0 [−2.4, +4.5]	.507	1.000
Capitals vs normal	+2.5 [+0.2, +4.9]	.039	.353
Example vs absent	+1.4 [−6.2, +9.1]	.676	1.000
Top vs bottom third	+3.8 [−1.3, +9.0]	.126	.884
Long vs short file	−3.7 [−8.2, +0.7]	.090	.720
Urgent vs no pressure	−4.3 [−14.1, +5.5]	.338	1.000

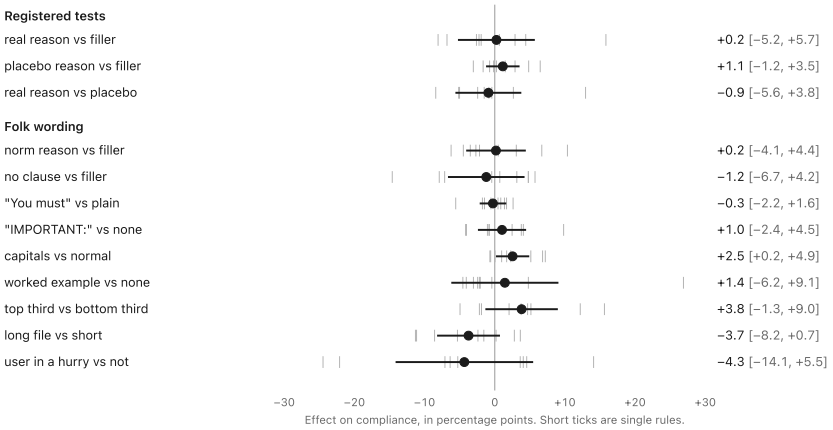


Figure 1: Study 1 effects with 95% intervals. Each short tick is one rule.

+5.35), and the prediction interval for a new rule is −11.57 to +11.87. The randomization test agrees on the average, at $p = .917$ for real and $p = .300$ for placebo. The real reason on `final_no_digits`, “because the voice assistant reading these reports aloud stumbles over numerals and identifiers”, is close to the vendor’s text-to-speech example, and it reads +2.9 (SE 4.0).

Placebo and the Langer analogue. A placebo is +1.1 [−1.2, +3.5]. A registered sensitivity, the TOST on real minus placebo at 5 points, passes at $p = .040$ and stops passing at a margin of 4.5 ($p = .058$). We do not read it as equivalence. Per-rule gaps run from −8.4 to +12.9, and because ceilings shrink realized gaps the test passed in 20.0% of pre-freeze simulations with a nominal real effect of 5 and a placebo of 0 (`docs/REVIEW-2-prereg.md`). The Langer analogue does not reproduce here. In that paradigm a placebic reason does what a real one does and the favor’s size decides whether either works. Here neither moved compliance detectably on average, and the moderator of reason by pressure, a loose analogue of the favor split, shows nothing (secondary, no significance claim): real −0.1 [−5.0, +4.8], placebo −1.3 [−5.3, +2.6], norm +0.4 [−7.5, +8.2].

Wording. No wording factor’s average across rules survives Holm (Table 3, Figure 1), and Section 8 reports the single rules. Position and length repeat the manipulations of McMillan [11]. Neither differs detectably, though top against bottom reaches +9.0, past the bound of about 6 points that work sets on its one rule. The registered sensitivity for run-level factors, shown minus unshown, gives long versus short −2.7 [−8.0, +2.7] and urgent versus none −2.7 [−13.9, +8.5].

Table 4: Study 2 family, Holm across the five, in percentage points (`results/NUMBERS.md`). The +0.0 rows are ties at a bound, and their zero-width intervals are not evidence of no effect.

Contrast	Points [95% CI]	p	Holm p
C1 trigger vs minimal, in view	+17.6 [+6.3, +29.0]	.006	.023
C2 directive vs trigger, in view	+0.0 [+0.0, +0.0]	1.000	1.000
C3 "Not for" clause, false loads, in view	+0.0 [+0.0, +0.0]	1.000	1.000
C4 `aa-` vs `zz-` at 300 skills	+51.3 [+37.7, +64.8]	$< .001$	$< .001$
C5 trigger vs minimal, 300 skills, `zz-`	−27.1 [−67.1, +13.0]	.154	.462

5 Study 2 and the measured listing rule

The listing rule. Claude Code tells the model which skills exist through a listing whose descriptions share a character budget, a limit the vendor documents [2]. We measured the exact rule behind it (`docs/LISTING-MECHANISM.md`). The budget depends on the model. In the CLI's code it is context window times characters per token times 0.01, and we read it to within 3 as 30,000 characters on Sonnet 5.5 and Haiku 5.5 and below 10,240 on `claude-sonnet-4-6`. If every entry fits, every skill keeps its description. Otherwise bundled skills keep theirs, every other skill is charged for its bare name, and the rest are sorted by usage score with ties in name order. The CLI walks that list once. A skill keeps its description if it fits what is left, and if it does not fit the walk continues, so a short description further down can still fit. Usage score is $usageCount \times \max(0.5^{d/7}, 0.1)$, with d the days since last use, so one call outranks every never-used skill.

So a library past the budget does not lose every long description. At 300 skills all five `aa-` wordings reached the model in 1.000 of runs, and earlier use also saves a description. What loses is a long description on a never-used skill that sorts after the budget is spent. A short one survives only if it fits the leftover, which at a late target's turn at 300 skills has a median of 24.5 characters in Study 2b (90th percentile 63, maximum 92, `results/listing_check.json`).

This rule is a property of Claude Code 2.1.303. It was measured with no model judgment, by comparing the entries that carry descriptions in the requests the CLI sent against what the rule predicts. Over 4,915 listing entries it made 0 errors, against 28 errors on the 2,590 pilot entries with usage removed. Eight variants gave the identical described set and order, which rules out relevance to the prompt, the tool list, model and context size among these models, creation order, modification time and nondeterminism. In a sample from a public library the budget binds between 100 and 150 skills (`results/listing_budget.json`). The decay term was read from the code in the shipped CLI and not tested live.

Design. Each run has one of 12 target skills, with a fixed body and one of 5 description wordings: minimal ("Reconciles ledgers."), padded (minimal plus a content-free sentence matching a trigger list's length), trigger ("Use when ..."), trigger_not (trigger plus "Not for ...") and directive ("ALWAYS invoke this skill when ... You MUST use it ..."). Library is 0, 50 or 300 distractors, at 25%, 25% and 50% of runs, sampled from 388 real skills of a public MIT-licensed library. Name prefix is `aa-` or `zz-`, an artificial stand-in for early and late. Prompts are keyword, generic, indirect or decoy, the last being a nearby request that does not need the skill. Each run's target has a unique name, so none starts with usage, though distractors invoked in one run do gain rank later. Version, model, effort and isolation are as in Study 1, with the Skill tool added and a limit of 6 turns. A load is a Skill call naming the target or a Read of its `SKILL.md` at any point in the run, decided by code. Contrasts are per target and averaged with equal weight, with a t interval across targets and Holm across the family.

Results. Of 600 planned runs, 598 were used. Two were excluded by the leak scan on the needle `agent-memory/`, which matches a public distractor's text. Load rates by prompt were 0.877 keyword, 0.660 generic, 0.752 indirect and 0.007 decoy.

C1 survives Holm (Table 4). With the description in view a trigger list loaded the skill in 65 of 65 non-decoy runs and one short sentence in 56 of 68 (`paper/counts.json`), an estimate of +17.6 points. "In view" pools libraries 0 and 50 with early names at 300, and that last stratum, past the budget, holds 7 of minimal's 12 misses (0.73 of 26 runs against 1.00 of 18). Strictly below the budget, post hoc, the contrast is +12.4 [+1.5, +23.2] in Study 2 and +3.5 [−10.0, +16.9] in Study 2b at 50 skills. Length alone does not explain C1, since padded versus minimal is −6.7 [−23.2, +9.8] as a secondary. Every wording loaded on every keyword prompt in view. The other registered secondaries, none corrected, put the gain where a

Table 5: Study 2b family, Holm across the four, in percentage points.

Contrast	Points [95% CI]	p	Holm p
D1 minimal vs trigger, 300 skills	+29.8 [+19.2, +40.3]	$< .001$	$< .001$
D2 short_trigger vs trigger, 300 skills	+29.4 [+20.5, +38.4]	$< .001$	$< .001$
D3 short_trigger vs minimal, 300 skills	−0.3 [−9.2, +8.6]	.939	.939
D4 short_trigger vs trigger, 50 skills	+2.1 [−2.5, +6.7]	.339	.678

prompt does not use the skill's own words, at +46.7 [+3.9, +89.4] on generic prompts (7 targets) and +12.5 [−17.1, +42.1] on indirect ones (8 targets). That placement does not repeat in Study 2b at 50 skills, where generic prompts loaded minimal in 14 of 14 runs and trigger in 13 of 14.

C2 and C3 sat at a ceiling and a floor. Directive loaded in 72 of 72 runs and trigger in 65 of 65. On decoy prompts the "Not for" wording falsely loaded in 0 of 18 runs and trigger in 0 of 24. These ties give zero-width intervals and a p of 1.000 from a rule in the code (Section 9), and they are not evidence of no effect, since the design left no room to see one. Across all 138 decoy runs there was one false load, on a minimal description in a library of 0.

C4 is +51.3 points, an average over very different gaps. Raw `aa-` minus `zz-` at 300 skills is +12.2 for minimal, +32.1 for padded, +76.5 for trigger, +76.2 for trigger_not and +64.0 for directive. C5 was too wide. Only 8 of 12 targets were estimable, and the interval of −67.1 to +13.0 cannot separate the wordings. The point estimate leaned toward reversal: at 300 skills with a late name minimal loaded at 0.61 in raw rate, trigger at 0.24 and directive at 0.36, and only minimal ever kept its description (0.65 of runs against 0.00). The registered secondary C1 minus C5 is +40.1 [+7.3, +72.9] on the 8 targets with every cell, the direction the listing rule predicts, with no correction. Study 2b was registered to settle this.

6 Study 2b: the reversal for a late, never-used skill

Design. We wrote this study after reading Study 2, and none of its data was seen when its hypotheses were set. All names are `zz-` and none repeats a Study 2 target. It adds one wording, short_trigger, a mechanical rewrite of the minimal text into trigger form ("Use when reconciling ledgers."). Mean lengths are 21 characters for minimal, 32 for short_trigger and 140 for trigger. The 12 targets are crossed with the 3 wordings, at 12 runs per cell at 300 distractors and 4 at 50, for 576 planned. Decoys are dropped. Three distractors invoked in Study 2 keep their rank, the same across arms.

Results. We used 534 runs. Another 41 were not run, because the registered six-point quota counter read the whole account while Study 1 ran at the same time, which the registration did not anticipate. The batch stopped at 535 of 576, and the registration says whatever exists is analyzed. One further run is excluded by the leak scan, a false positive on the needle `agent-memory/`.

Both registered readings fired (Table 5). The code returned, "past the budget a short plain description beats a long trigger list; a short description that says when to use the skill beats a long one past the budget and costs under 10 points below it." In this design "past the budget" means a never-used `zz-` target at 300 skills. It does not extend to a long description that keeps its text (Section 5). D1 repeats the direction of C5 with a usable interval. D3 cannot separate the two short wordings, since its interval holds gaps of about 9 points either way. D4's lower bound of −2.5 clears the registered floor of −10, so the claim is a cost under 10 points. The three cells at 50 skills sit near the ceiling (Table 6), and the interval comes from one miss: 11 of 12 targets tie at zero and one reads +25. By run, a Newcombe interval is −6.1 to +11.3 and the Clopper-Pearson worst case is −8.0 (`docs/REVIEW-3-final.md`). Figure 2 shows the crossing in both studies. The trigger wording never kept its description at 300 skills here, and it still loaded in 0.211 of runs with only its name in the listing. The direction is the same in all three prompt types, and no interaction was tested. Within keyword, generic and indirect prompts, as registered secondaries with no correction, D1 is +32.1, +24.5 and +28.8 and D2 is +31.8, +25.0 and +28.0, on 11 or 12 targets each (`results/NUMBERS.md`), and D1's intervals are 26.3 to 48.3 points wide.

7 Study 2c: raising the budget

Design. No arm of Studies 2 or 2b changed the budget, so "past the budget" always meant "300 skills", and whether a description fit depended on the distractors a run drew. Study 2c moves the budget and nothing else. We wrote it after reading Studies 2 and 2b, and none of its data was seen when its hypotheses were set. Every run has 300 distractors and one of the 12 targets of Study 2b under a `zz-` name with no usage, and

Table 6: Study 2b by library and wording. "Reached model" is the share of runs whose request held the target's description. The last two columns are exploratory, because whether a description fits depends on the distractors a run drew and is not randomized.

Library	Wording	Load rate	n	Reached model	Load if reached	Load if not
50	minimal	0.953	43	1.000	0.953	
50	short_trigger	1.000	44	1.000	1.000	
50	trigger	0.978	46	1.000	0.978	
300	minimal	0.511	137	0.526	0.750	0.246
300	short_trigger	0.500	136	0.338	0.848	0.322
300	trigger	0.211	128	0.000		0.211

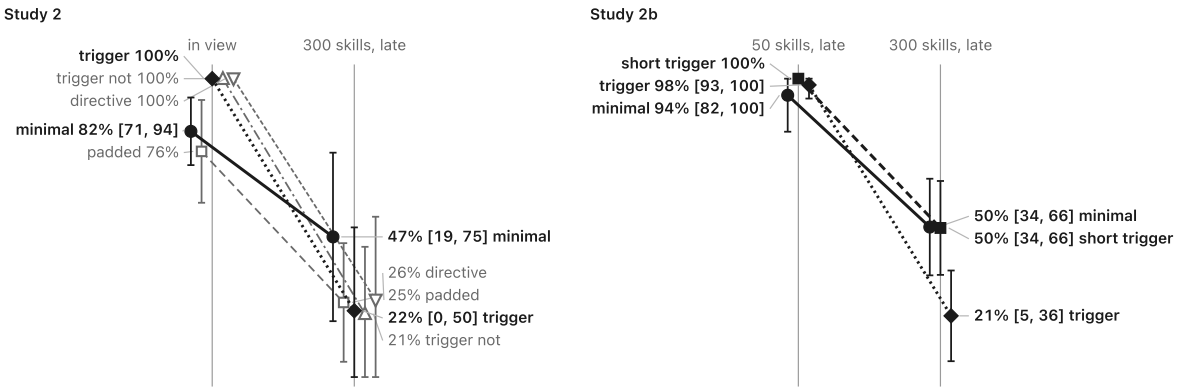


Figure 2: Load rate by wording, as the mean across targets with equal weight and its 95% interval across targets, cut at 0 and 100. Left, Study 2, where in view pools libraries 0 and 50 with early names at 300, and some late cells have under 12 targets. Right, Study 2b.

new run numbers keep every name unique. The wordings are minimal and trigger. The budget is the CLI's own 30,000 characters or 100,000 through the documented variable `SLASH_COMMAND_TOOL_CHAR_BUDGET` [2], and any 300 from the library need at most about 79,000. At 8 runs per target, wording and budget there are 384 runs, with keyword, generic and indirect prompts drawn independently. Study entries were cleared from the account's usage record before the pilot, so distractors carry usage only from the pilot and from earlier runs of this batch, in both arms. Version, model, effort, tools, turn limit, isolation and outcome are as in Study 2b. A 16-run pilot read only status, cost, turns, leak flags and whether the description reached the model, and no load. If under 95% of raised runs carried the description, the registered reading was that the manipulation failed and nothing is read.

Results. All 384 runs were used, with no errors, leaks, timeouts or turn-limit runs. The manipulation check read 1.000 in raised runs. All four contrasts pass Holm (Table 7). The code returned, "the budget causes the reversal: with the budget raised a long trigger list recovers and is within 10 points of a short description or above it."

E1 is the recovery. E2 is smaller as predicted, and it is not zero, because minimal kept its description in only 0.552 of runs at the CLI's budget (Table 8). E3 says the reversal closes and the long list leads again, at about the size of C1, though its lower bound is +0.7. E4 is the reversal of the reversal. As a registered secondary with no correction, minimal minus trigger at the CLI's own budget repeats D1 at +24.0 [+13.0, +34.9]. E1 keeps its direction within keyword (+50.7 [+25.9, +75.5], 12 targets), generic (+86.7 [+71.0, +102.4], 10 targets) and indirect prompts (+81.5 [+52.5, +110.4], 9 targets), also with no correction and no interaction test, and a t interval can pass 100 points.

Cost per run more than doubles, from about $0.06 to $0.134, with every description in every request and more turns per run. The share of runs that load only another skill stays small, at 0.052 and 0.000 against 0.021 at the CLI's budget, and the share that load no skill falls (Table 8).

What this shows and does not. With targets, library, names and prompts fixed, moving the budget alone reverses the reversal, so at 300 skills a short description's advantage comes from reaching the model, and a long list's comes back once it does. Raising the budget shows every distractor's description as well as

Table 7: Study 2c family, Holm across the four, in percentage points (`results/NUMBERS.md`).

Contrast	Points [95% CI]	p	Holm p
E1 trigger, raised vs the CLI's budget	+69.8 [+53.7, +85.8]	< .001	< .001
E2 minimal, raised vs the CLI's budget	+29.2 [+6.1, +52.2]	.018	.035
E3 trigger vs minimal, raised	+16.7 [+0.7, +32.7]	.043	.043
E4 interaction, E3 minus trigger vs minimal at the CLI's budget	+40.6 [+22.7, +58.6]	< .001	.001

Table 8: Study 2c by budget and wording, at 300 skills with a late name. The interval is across targets and cut at 1. Other skill only and no skill are shares of runs, and cost is at API-equivalent prices.

Budget	Wording	Load rate [95% CI]	n	Reached model	Other skill only	No skill	Cost per run	Turns
CLI's own	minimal	0.531 [0.38, 0.68]	96	0.552	0.021	0.448	$0.061	4.19
CLI's own	trigger	0.292 [0.12, 0.46]	96	0.000	0.021	0.688	$0.060	3.71
Raised	minimal	0.823 [0.67, 0.98]	96	1.000	0.052	0.125	$0.134	4.84
Raised	trigger	0.990 [0.97, 1.00]	96	1.000	0.000	0.010	$0.134	5.14

the target's, which is what a person who raises it gets, so this tests the budget and not the target's description alone. The size belongs to a library of this size and to `zz-` names with no usage, and the rule and the variable belong to this CLI version. We used the variable and did not test `skillListingBudgetFraction`.

8 Exploratory results

None of this section is a registered test. Its numbers come from `results/exploratory.json` and `results/robust.txt`. Of the exploratory list in `PREREG.md`, relative depth, rule by task, paraphrase variation, two-way wording interactions and spillover were not run, and neither cost class nor polarity was tested.

Single rules and factors. Nobody registered these cells, and they run across rule and factor post hoc (Table 9). Two of the 72 cells on rule-level factors survive Holm: a worked example on `final_no_first_person` and `IMPORTANT:` on `reverse_imports`. Adding top versus bottom, long versus short and urgent versus none on each rule makes 99 cells, of which seven survive. So the averages near zero in Table 3 hide some large effects on single rules. The example adds specification as well as form, which fits Razmara et al. [12], and it has no length-matched control. The real reason's +15.8 does not survive either correction, and we do not read it as evidence that a real reason works on that rule.

Which rule it is. Focal compliance runs from 0.09 to 0.93 by rule, with an SD of 0.29 (Table 10). The pool rule admitted rules between 0.10 and 0.95 in pilots, so selection set this range by construction, and it is not an estimate over rules in general. Rule identity is not randomized and is tangled with how strict each checker is, so a writer cannot treat it as a lever.

Having a rule, and restating it. Shown minus no rule is +54.2 [+33.1, +75.2] points. The no-rule baseline is not a clean "no instruction" state, because other rules are present. Named in the prompt minus shown in the file is +7.1 [+0.2, +13.9] points, higher on 8 of 9 rules. This rests on 3 to 11 runs per rule, so it is a lead for a registered test and nothing firmer.

The mixed model. The registered robustness model (`analysis/robust.R`) decides nothing. Its fit is singular, with a run SD and a placebo slope SD of 0. It gives prediction intervals for a new rule of −5.7 to +5.1 points for real and −1.6 to +4.4 for placebo, built from a slope SD of 2.18 points from that fit and a 1.96 quantile where nine rules call for a t quantile. The random-effects interval in Section 4 is about twice as wide and is the one to read. The model has rule slopes only on real and placebo, so its t values for short files (3.93) and urgency (−4.23) are not claims about new rules, and those two contrasts do not pass under the registered estimator.

Where calls go at 300 skills. In Study 2b, with the target's description gone, the shares are 0.254 target loaded, 0.025 other skill only and 0.721 no skill (n 283). With the description kept they are 0.788, 0.034 and 0.178 (n 118). Losing the description goes with using no skill and not with using a wrong one. Fit is not randomized within a wording, and this split mixes wordings and targets. Study 2c, where the budget is randomized, is the causal test.

Replication on Haiku. Registered as 300 runs of the same design on `claude-haiku-5-5`, read for sign only, with no significance claim, on a pool selected on Sonnet. We used 299, one being excluded by the leak

Table 9: Rule-by-factor cells, exploratory and post hoc, in points with run-clustered SE (`results/exploratory.json`). The first two rows pass Holm across 72 cells. "Outside" marks a run-level or position contrast, which only the 99-cell family holds.

Factor	Rule	Points (SE)	Raw p	Holm, 72	Holm, 99
Worked example vs none	`final_no_first_person`	+26.9 (3.4)	$< .001$	$< .001$	$< .001$
`IMPORTANT:` vs none	`reverse_imports`	+9.8 (2.8)	$< .001$	.032	.041
Urgent vs no pressure	`one_blank`	−24.5 (3.4)	$< .001$	outside	$< .001$
Real reason vs filler	`final_no_first_person`	+15.8 (5.6)	.004	.312	.401
Capitals vs normal	`final_no_contractions`	+7.2 (2.7)	.007	.481	.614
No clause vs filler	`func_lines`	−14.6 (5.7)	.011	.744	.951

Table 10: Focal rules and compliance by rule, across all wordings. Named in the prompt rests on 3 to 11 runs per rule.

Rule (bare wording)	Shown	No rule	Named	n shown
Do not use contractions or apostrophes in your final message.	0.87	0.28	1.00	786
Do not use digits in your final message; spell numbers out.	0.09	0.00	0.14	786
Do not use the words I, my, we or our in your final message.	0.32	0.00	0.38	786
Keep every Python function you write, including its signature, to 8 lines or fewer.	0.44	0.00	0.30	786
Keep every line of Python you write to 60 characters or fewer.	0.81	0.00	1.00	785
Give every variable, parameter and loop variable you write a name of at least 4 letters.	0.50	0.04	0.62	786
Do not leave two blank lines in a row in any Python you write.	0.41	0.00	0.50	786
Sort the import lines of every new Python file in reverse alphabetical order.	0.84	0.00	0.91	786
Create new Python files with your Write tool, not through shell redirection.	0.93	0.00	1.00	785

scan, because a tool result named the CLI's folder for oversized output on the operator's machine. Real versus filler is −0.9 [−5.2, +3.4] and placebo versus filler is +0.3 [−3.3, +3.9]. The signs are mixed and both sit near zero, so neither favors a reason. The frozen `reading()` returns its practical-null row on this batch. We do not claim that, because the registration reads Haiku for sign only.

9 Deviations, disclosures and timing

All of these are logged in `docs/DEVIATIONS.md` with date and reason. The registered analysis is still what is reported.

All studies. A bug in the frozen analysis was found on first reading Study 2's output and fixed. `by_rule` returned a NaN p when every rule or target agreed exactly, and the NaN corrupted Holm for the rest of the family, so C4 at +51 points first showed an adjusted p of 1.000. A zero standard error now gives $p = 1$ for a zero estimate and $p = 0$ otherwise. It changes results only where such a tie exists. The confirmatory batch logs are tracked, with 1,000, 600, 535 and 384 lines, all of them `ok`, so no run was rerun.

Timing. We rewrote the git history once, on 2 November 2026 and before any public release of the repository, to remove the account and device ids that sample captures kept. `git filter-repo` removed that one `metadata` field from every kept capture in every commit, so the commit and tag hashes changed. Dates, messages and every other file are as they were, and the preregistration files hash to the same git blobs at the new tags as at the old ones (`PREREG.md` 2f7f2ff909, `PREREG-STUDY2.md` e6002afada, `PREREG-STUDY2B.md` 4290c9823c). The Study 2c tag came after the rewrite. Two third-party time points exist, and a reader can verify both once the repository is public. GitHub reports the repository created at 03:52:50 UTC on 2 November 2026, one second after the Study 1 freeze commit date. The push of the Study 2c tag's commit is timed by a GitHub Actions run created at 2026-11-02T07:46:31Z, before the study's first run, whose first log line is at 07:46:48 UTC. For the rest, the order of events rests on the commit dates, which a reader cannot fully verify from outside. No timestamping service was used.

Study 1. (1) `git add -A` at the Study 2 tag committed about 154 partial Study 1 runs before anyone read a contrast, and the final data commit supersedes them. (2) The audit sampler as frozen drew from every shown rule, so it was changed to draw 100 focal observations as registered, and it also gives the reader a diff. (3) With about 260 runs in and before any contrast, an early blind audit of 36 extra observations agreed with 34 and found one fault. `write_tool` failed a run for `cat »` onto a file the repository already had, so the checker now ignores a redirect onto a file that existed at the base commit, and all runs were rescored. (4) Harness edits after the tag touch no run's behavior in Studies 1, 2 or 2b: a narrower leak

needle, a `probe` and an `env` parameter, factored helpers, kept samples without `metadata`, a wrapper shell written once, `RUN_ROOT` and `TMPDIR` configurable with the frozen values as their defaults on a Mac, and a dry-run flag. The batch ran the code as frozen plus the needle and the `probe`. (5) After the registered audit and before any contrast, the scope of "you write" was written down as in Section 3, with no checker changed. (6) The audit used three readers. (7) The mixed model's export was restricted to focal rules.

Studies 2b and 2c. The Study 2b batch stopped at 535 of 576 on its registered quota stop (Section 6). Study 2c has one additive edit after its tag and before any outcome was read: each cell of its grid gained a mean across targets and its interval, so the figure can draw them. No family row, secondary or reading changes, which we checked by running the frozen and edited code on a synthetic batch.

Registered secondaries computed late. The frozen analyses omitted secondaries the Study 2 and 2b registrations ask for (C1 minus C5, prompt type by wording, contrasts within prompt types). We added them on 2 November 2026, after reading the family results, in the form the registrations describe, with no correction, and they decide nothing.

Disclosures from the pre-freeze review (`docs/REVIEW-2-prereg.md`). PRESSURE is an instruction as well as pressure, since "keep it minimal and skip anything not strictly needed" can shorten code with no rule in play, and favor size in Langer maps to rule cost more than to user urgency. Rules interact mechanically: wrapping for `line_60` adds lines against `func_lines`, and long names from `name_length` do too. A clause on one rule may spill onto another. In the fielded files 60% of shown rules carry "because" against a census rate of 26%, so a single clause may be diluted relative to a file where it stands out. Rules near the top of the band leave little room, and selection brings some regression to the mean.

10 Limits

Scope. We ran one model, one CLI version, nine rules and two synthetic repositories for Study 1, and 12 targets written by us for Studies 2, 2b and 2c. The tool list is shorter than the default, which changes the system prompt from what an interactive user sees. Wording effects vary by model in the panel of Zhan et al. [20], so a null here may be specific to this model.

Nine rules is few. The by-rule interval speaks to new rules like these, drawn from the same kind of pool, and with $df = 8$ it rests on a small sample of stimuli. Realized power for a true 5-point effect is 0.34 to 0.56, so a real effect of that size could easily read as this study read.

What a reader should not conclude. Do not conclude that reasons never matter. The average interval allows up to 5.7 points, a new rule's prediction interval is about 12 points either way, and rules in conflict with repository precedent were not tested. Do not conclude that reasons fail to generalize, since we scored literal compliance and the vendor's claim is about going past it. Do not conclude that reasons are useless in a file, since people read these files too and we measured only the agent. Do not conclude that emphasis has no effect in general, since Claude Code's preamble already supplies IMPORTANT and MUST in every run. Do not read the zero rows of Study 2 as showing that "ALWAYS invoke" or a "Not for" clause does nothing, since both arms sat at a bound. Do not read Study 2b as saying short descriptions are always better. Where a long list fits it can win, as C1 and E3 show. At 50 skills the data bounds the short form's cost under 10 points near the ceiling, and it cannot see a cost of a few points. Past the budget the result depends on a CLI rule that another version can change.

Skills. The `aa-` and `zz-` prefixes are artificial. A target's name stays informative with no description, so a load past the budget can come from the name alone. Distractors come from one public library. short_trigger is a mechanical rewrite, and a person writing one by hand might do better. Every target started with no usage history, and by the listing rule one earlier call would change its standing, so nothing here describes a skill already in use. Only project skills and built-ins load here, because the harness passes project settings only. Small synthetic repositories may also read as tests, and thinking text is empty in the raw logs, so demand effects cannot be audited there.

11 Replication package

Code, data and the four frozen registrations are in the repository at `https://github.com/abhaymettu/because`, under an MIT license. Each confirmatory batch is under `data/`. Study numbers come from the generated `results/NUMBERS.md` or from the result file named beside them. From the repository root, `git tag -l 'prereg-*'` lists the freezes. `make test` and `make replay` need no subscription. `make reproduce`

regenerates the registered results, the exploratory block, the listing check, the figures of `docs/figures`, the Study 2 cell counts and the numbers page from the traces. `make calibration` and `make robust` take longer and write their own files. `make isolation` and the clause ratings need a login.

None of the 4,915 listing entries can be rerun offline from a clean checkout. The 2,590 pilot entries need 16 captures that hold full requests and are not in the repository, and the other 2,325 come from live runs, which `python -m analysis.listing_mechanism` repeats with 29 model calls and a login. The offline check is a narrower test on more runs. `python -m analysis.listing_check data/study2 data/study2b data/study2c` predicts from the manifests and fixtures whether each run's target kept its description, one target per run across 1,519 runs, and it matches 600 of 600, 535 of 535 and 384 of 384, against 583, 426 and 361 with usage removed (`results/listing_check.json`). `make rerun` repeats the runs into `data/rerun`, never over the committed traces, and needs Claude Code 2.1.303, which it checks, and a subscription. At API-equivalent prices Study 1 cost $102.84 (mean 9.3 turns, 33.5 s per run), Study 2 cost $26.97, Study 2b cost $29.52 and Study 2c cost $37.35.

References

[1] Anthropic. Prompting best practices. Claude Platform documentation, current page, `https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices`, read 2 November 2026, .

[2] Anthropic. Extend Claude with skills. Claude Code documentation, current page, `https://code.claude.com/docs/en/skills`, read 2 November 2026, .

[3] Herbert H. Clark. The language-as-fixed-effect fallacy: A critique of language statistics in psychological research. *Journal of Verbal Learning and Verbal Behavior*, 12(4):335 to 359, 1973. doi: 10.1016/S0022-5371(73)80014-3.

[4] Valerie S. Folkes. Mindlessness or mindfulness: A partial replication and extension of Langer, Blank, and Chanowitz. *Journal of Personality and Social Psychology*, 48(3):600 to 604, 1985. doi: 10.1037/0022-3514.48.3.600.

[5] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6:65 to 70, 1979. doi: 10.2307/4615733.

[6] Inferal. Why AI agents ignore your instructions. Blog post, undated, `https://inferal-com.pages.dev/blog/instruction-adherence-research`, read 2 November 2026.

[7] Charles M. Judd, Jacob Westfall, and David A. Kenny. Treating stimuli as a random factor in social psychology: A new and comprehensive solution to a pervasive but largely ignored problem. *Journal of Personality and Social Psychology*, 103 (1):54 to 69, 2012. doi: 10.1037/a0028347.

[8] Ellen J. Langer, Arthur Blank, and Benzion Chanowitz. The mindlessness of ostensibly thoughtful action: The role of "placebic" information in interpersonal interaction. *Journal of Personality and Social Psychology*, 36(6):635 to 642, 1978. doi: 10.1037/0022-3514.36.6.635.

[9] Andrew Lauziere, Jonathan Daugherty, and Taisa Kushner. A regression framework for understanding prompt component impact on LLM performance. arXiv:2603.26830, 2026.

[10] Jinlin Liu, Wei Chen, and Xiaojin Zhang. Lost in implicit routing: How LLM agents fail to load relevant skills at scale. arXiv:2610.18522, 2026.

[11] Damon McMillan. Instruction adherence in coding agent configuration files: A factorial study of four file-structure variables. arXiv:2605.10039, 2026.

[12] Parsa Razmara, Seyedarmin Azizi, and Ardalan Aryashad. One skill takes over: Clashing agent skills and a one-file fix. arXiv:2610.25064, 2026.

[13] Donald J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657 to 680, 1987. doi: 10.1007/BF01068419.

[14] Scott Spence. Measuring Claude Code skill activation with sandboxed evals. Blog post, `https://scottspence.com/posts/measuring-claude-code-skill-activation-with-sandboxed-evals`, 2026.

[15] Haochuan Kevin Wang. Advertising to agents: Measuring description-framing bias in LLM tool selection. arXiv:2605.23916, 2026.

[16] Jiajie Wang, Yutong Zhao, Tianlin Li, Huashan Chen, Jinfu Chen, Kebin Peng, and Sen He. Agent skill evolution: How revisions affect coding agents. arXiv:2610.04832, 2026.

[17] Xiaonan Xu and Wenjing Wu. Stale instruction files override the code in coding agents: A one-sentence fix. arXiv:2610.30571, 2026.

[18] yurukusa. CLAUDE.md under test: What Claude Code rules actually do. Leanpub book, public description only, `https://leanpub.com/claude-md-under-test`, read 2 November 2026, 2026.

[19] Pawel Zarzycki. What open-source projects tell their coding agents: A census of AGENTS.md and CLAUDE.md. arXiv:2610.31715, 2026.

[20] Qishi Zhan, Seoyeon Jang, Zihan Dong, Minxuan Hu, Ziheng Chen, and Tonghui Qu. When verifiable counts depend on wording: Auditing wording robustness in instruction following. arXiv:2610.05278, 2026.

[21] Shuyang Zhang. What does a skill actually do? Estimands and evaluation validity for tool and skill use in LLM agents: A critical review. arXiv:2609.33153, 2026.

[22] Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Guardrails beat guidance: A large-scale study of rules, skills, and persistent configuration for coding agents. arXiv:2604.11088, 2026.

[23] Haichao Zhu, Geyang Xu, Zhaorui Yang, Jiyuan Wang, and Qian Zhang. Precedent or intent? What coding agents follow when evolving existing code. arXiv:2610.38244, 2026.